CONSTRUCTOR

The **init** function runs automatically when an object is created.

In Python, **self** represents the instance of a class. It is a parameter used in instance methods to access and modify the specific attributes and methods of an object created from that class.

```
class ATM:
    def __init__(self):  # DEFAULT CONSTRUCTOR OR INITIALIZER
        self.pin = None
        self.balance = 0
        print("ATM system initialized!")
        self.menu()  # Automatically call menu

    def menu(self):
        print("Hello! How can I help you?")
        print("1. Set PIN")
        print("2. Change PIN")
        print("3. Check Balance")
        print("4. Withdraw Cash")
```

```
atm1 = ATM()
```

```
ATM system initialized!
Hello! How can I help you?
1. Set PIN
2. Change PIN
3. Check Balance
4. Withdraw Cash
```

```
class Addition:

# INITIQALIZER
  def __init__(self):
# we are initializing INSTANCE variable
    self.num1 = 1000
    self.num2 = 2000
    self.num3 = 3000

  def result(self):
    X=10  # LOCAL variable
    self.num = self.num1 + self.num2 + self.num3+X
    print("Output", self.num)

# Here we create the object for call
Sum = Addition()
```

```
# calling the instance method using the object Sum
Sum.result()
```

```
Output 6010
```

```
class Demo:
    x = 100  # CLASS variable
a = Demo()
b = Demo()

print(a.x)  # 100
print(b.x)
```

```
100
100
```

```
class Student:
# Defining a parameterized constructor having arguments
  def __init__(self, name, ids, college):
    print("This is a parmeterized costructor in python")
    self.name = name
    self.ids = ids
    self.college = college

  def Display_Details(self):
    print("Student Details")
    print("Student Name", self.name)
    print("Student ids", self.ids)
    print("Student college", self.college)

# Here we create the objects
student = Student("JOHN", 2023, "MIT")
student1=  Student("rahul", 2023, "MIT")

# calling the instance method using the object student
student.Display_Details()
student1.Display_Details()
```

```
This is a parmeterized costructor in python
This is a parmeterized costructor in python
Student Details
Student Name JOHN
Student ids 2023
Student college MIT
Student Details
Student Name rahul
Student ids 2023
Student college MIT
```

```
class Student:
  def __init__(self, name, marks):
    self.name = name
    self.marks = marks

  def display(self):
    print("Name:", self.name)
```

```
    print("Marks:", self.marks)
# Main Program
s1 = Student("Rahul", 85)
s1.display()
```

```
Name: Rahul
Marks: 85
```

```
class Employee:
  def __init__(self, name="Not Assigned", salary=0):
    self.name = name
    self.salary = salary

  def display(self):
    print(self.name, self.salary)

e1 = Employee()
e2 = Employee("Amit", 40000)
e1.display()
e2.display()
```

```
Not Assigned 0
Amit 40000
```

```
class BankAccount:
  def __init__(self, name, balance):
    self.name = name
    self.balance = balance

  def deposit(self, amount):
    self.balance += amount

  def withdraw(self, amount):
    self.balance = self.balance- amount

  def display(self):
    print("Account Holder:", self.name)
    print("Balance:", self.balance)

acc = BankAccount("Sneha", 10000)
acc.deposit(2000)
acc.withdraw(1500)
acc.display()
```

```
Account Holder: Sneha
Balance: 10500
```

Overloading and Overriding

Method Overloading is a type of Compile time Polymorphism using which we can define two or more methods in the same class with same name but different parameter list.Python doesn't support traditional overloading, similar functionality can be achieved using default arguments and variable-length arguments.

Method Overriding is a type of Run time polymorphism.A child class methods overrides the parent class method of the same name,parameters and return type.It is used to over-write a parent class method in the derived class.

Double-click (or enter) to edit

SINGLE INHERITANCE

```
# Base class
class Parent:
    def func1(self):
        print("This function is in parent class.")

# Derived class
class Child(Parent):
    def func2(self):
        print("This function is in child class.")

# Driver code
obj = Child()
obj.func1()
obj.func2()
```

```
This function is in parent class.
This function is in child class.
```

When a class inherits from more than one base class, it is called multiple inheritance.
The derived class inherits all features of its base classes.

MULTIPLE INHERITANCE

```
# Base class 1
class Mother:
    mothername = ""

    def mother(self):
        print(self.mothername)

# Base class 2
class Father:
    fathername = ""

    def father(self):
        print(self.fathername)

# Derived class
```

```
class Son(Mother, Father):
    def parents(self):
        print("Father :", self.fathername)
        print("Mother :", self.mothername)

# Driver code
s1 = Son()
s1.fathername = "RAM"
s1.mothername = "SITA"
s1.parents()
```

```
Father : RAM
Mother : SITA
```

```
class Class1:
    def m(self):
        print("In Class1")

class Class2(Class1):  # OVERRIDING IN 2 CLASSES
    def m(self):
        print("In Class2")

class Class3(Class1):   # OVERRIDING IN 2 CLASSES
    def m(self):
        print("In Class3")

class Class4(Class2, Class3):
    pass

obj = Class4()
obj.m()
```

```
In Class2
```

```
class Class1:
    def m(self):
        print("In Class1")

class Class2(Class1):
    pass

class Class3(Class1):  # OVERRIDING IN ONE CLASS
    def m(self):
        print("In Class3")

class Class4(Class2, Class3):
    pass

obj = Class4()
obj.m()
```

```
In Class3
```

```
class Class1:   #Calling methods of parent classes from child class
    def m(self):
        print("In Class1")

class Class2(Class1):
    def m(self):
        print("In Class2")
        Class1.m(self)

class Class3(Class1):
    def m(self):
        print("In Class3")
        Class1.m(self)

class Class4(Class2, Class3):
    def m(self):
        print("In Class4")
        Class2.m(self)
        Class3.m(self)

obj = Class4()
obj.m()
```

```
In Class4
In Class2
In Class1
In Class3
In Class1
```

Super Function

Super Function in Python is used to call a method from a parent (base) class, especially in multiple inheritance. It helps avoid explicitly naming the parent class

```
class Class1:
    def m(self):
        print("In Class1")

class Class2(Class1):
    def m(self):
        print("In Class2")
        super().m()

class Class3(Class1):
    def m(self):
        print("In Class3")
        super().m()

class Class4(Class2, Class3):
    def m(self):
        print("In Class4")
```

```
        super().m()

obj = Class4()
obj.m()
```

```
In Class4
In Class2
In Class3
In Class1
```

MULTILEVEL INHERITANCE

```
# Base class
class Grandfather:
    def __init__(self, grandfathername):
        self.grandfathername = grandfathername

# Intermediate class
class Father(Grandfather):
    def __init__(self, fathername, grandfathername):
        self.fathername = fathername
        # Call the constructor of Grandfather
        Grandfather.__init__(self, grandfathername)

# Derived class
class Son(Father):
    def __init__(self, sonname, fathername, grandfathername):
        self.sonname = sonname
        # Call the constructor of Father
        Father.__init__(self, fathername, grandfathername)

    def print_name(self):
        print('Grandfather name :', self.grandfathername)
        print('Father name :', self.fathername)
        print('Son name :', self.sonname)

# Driver code
s1 = Son('ABC', 'MUKESH', 'DHIRUBHAI')
print(s1.grandfathername)
s1.print_name()
```

```
DHIRUBHAI
Grandfather name : DHIRUBHAI
Father name : MUKESH
Son name : ABC
```

Hierarchical Inheritance is a type of inheritance in which a single base class is inherited by multiple derived classes. In this scenario, each derived class shares common attributes and methods from the same base class, forming a hierarchy of classes.

HIERARCHICAL INHERITANCE

```
# Base class
class Parent:
    def func1(self):
        print("This function is in parent class.")

# Derived class 1
class Child1(Parent):
    def func2(self):
        print("This function is in child 1.")

# Derived class 2
class Child2(Parent):
    def func3(self):
        print("This function is in child 2.")

# Driver code
object1 = Child1()
object2 = Child2()

object1.func1()
object1.func2()
object2.func1()
object2.func3()
```

```
This function is in parent class.
This function is in child 1.
This function is in parent class.
This function is in child 2.
```

```
class Shape:
    def __init__(self, color):
        self.color = color

    def area(self):
        pass

class Circle(Shape):
    def __init__(self, color, radius):
        super().__init__(color)
        self.radius = radius

    def area(self):
        return 3.14 * self.radius**2

class Square(Shape):
    def __init__(self, color, side_length):
        super().__init__(color)
        self.side_length = side_length

    def area(self):
        return self.side_length**2
```

```
# Usage
circle = Circle("Red", 5)
square = Square("Blue", 4)
print(circle.area())
```

```
78.5
16
```

HYBRID INHERITANCE

```
# Base class
class School:
    def func1(self):
        print("This function is in school.")

# Derived class 1 (Single Inheritance)
class Student1(School):
    def func2(self):
        print("This function is in student 1.")

# Derived class 2 (Another Single Inheritance)
class Student2(School):
    def func3(self):
        print("This function is in student 2.")

# Derived class 3 (Multiple Inheritance)
class Student3(Student1, School):
    def func4(self):
        print("This function is in student 3.")

# Driver code
obj = Student3()
obj.func1()
obj.func2()
```

```
This function is in school.
This function is in student 1.
```